

CS 3006

Parallel and Distributed Computing

Lecture 6

Danyal Farhat
FAST School of Computing
NUCES Lahore

Principles of Parallel Algorithm Design, Task Dependency Graphs, Granularity and Concurrency, and Task Interaction Graphs

Lecture's Agenda

- **Parallel Algorithm Design Life Cycle**
- **Parallel Computing Example**
- **Task Decomposition**
- **Task Dependency Graph**
- **Granularity**
 - **Fine Grained**
 - **Coarse Grained**

Lecture's Agenda (Cont.)

- **Concurrency**
 - **Maximum Degree of Concurrency**
 - **Critical Path and Critical Path Length**
 - **Average Degree of Concurrency**
- **Task Interaction Graphs**
- **Processes and Mapping**
- **Summary**
- **Additional Resources**

Steps in Parallel Algorithm Design

- **Identification:** Identifying **portions** of the work that can be performed concurrently.
 - Work-units are also known as **tasks**
 - E.g., Initializing two mega-arrays are two tasks and can be **performed** in parallel
- **Mapping:** The process of mapping **concurrent** pieces of the work or tasks onto multiple processes running in parallel.
 - Goal: balance load; **maximize** data locality
 - Approach: **static vs. dynamic** task assignment
 - Process is **logical agent** for computation over a physical processing element (processor).

Steps in Parallel Algorithm Design (Cont.)

- **Data Partitioning:** Distributing the input, output, and intermediate data associated with the program.
 - One way is to copy whole data at each processing node
 - ✓ Memory challenges for huge-size problems
 - Other way is to give fragments of data to each processing node
 - ✓ Communication overheads
- **Defining Access Protocol:** Managing accesses to data shared by multiple processors
 - Access protocol that manage communication and synchronization

Parallel Computing Example

Chess Player

- A parallel program to play chess might look at all the **possible first moves** it could make
- Each different first move could be **explored** by a different processor, to see how the game would continue from that point
- Results have to be **combined** to figure out which is the best first move
- Famous **IBM Deep Blue** machine that beat Kasparov
 - Brute force **computing** power
 - Massively parallel with **30 nodes**, with each node containing a 120 MHz P2SC microprocessor

Task Decomposition

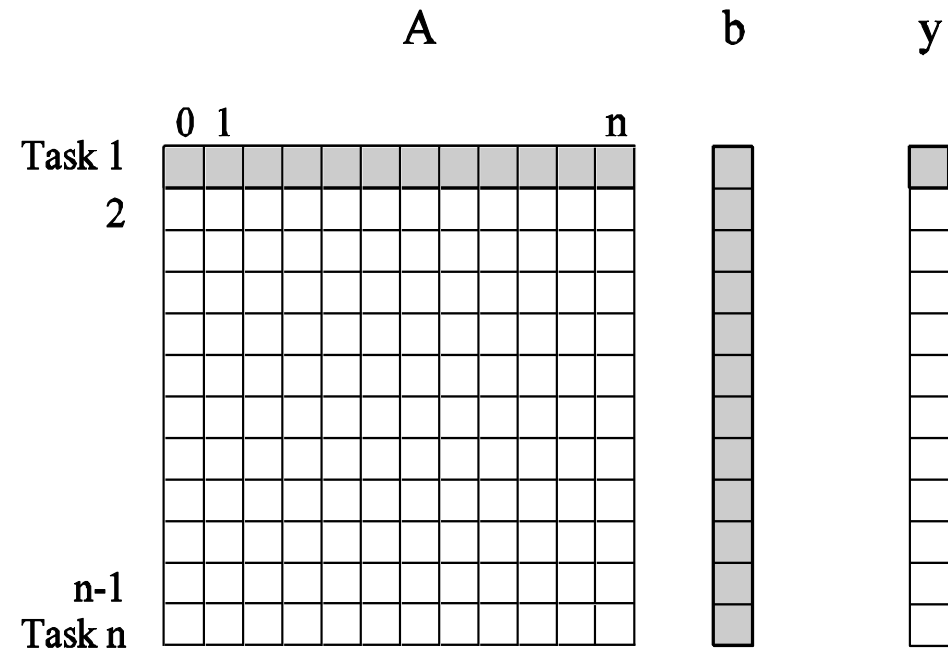
Decomposition

- “The process of **dividing** a computation into smaller parts, some or all of which may potentially be executed in parallel.”

Tasks

- Programmer-defined **units of computation** into which the main computation is subdivided by means of decomposition
- Tasks can be of **arbitrary size**, but once defined, they are regarded as indivisible units of computation
- Simultaneous execution of multiple tasks is the **key** to reducing the time required to solve the entire problem

Multiplication of a Dense Matrix with a Vector



- Problem can be **decomposed** into n tasks
- Computation of each element of vector y is **independent** of other elements
- No control dependencies so **no task-dependency graph**

Vector Multiplication $n \times 1$

- So the sequential multiplication **program** like:

```
for (row = 0; row < n; row++)
```

```
    y[row] = dot_product( get_row(A, row), get_col(b));
```

- Can be **transformed** to parallel program:

```
for (row = 0; row < n; row++)
```

```
    y[row]=  create_thread ( dot_product(get_row(A, row), get_col(b)));
```

Matrix Multiplication $n \times n$

Sequential:

```
for (row = 0; row < n; row++)
    for (column = 0; column < n; column++)
        c[row][column] = dot_product( get_row(a, row), get_col(b, col));
```

Multithreaded:

[illegible]

Task Dependency Graph

- The tasks in the previous examples are **independent** and can be performed in any sequence.
- In most of the problems, there exist some sort of **dependencies** between the tasks.
- An **abstraction** used to express such dependencies among tasks and their relative order of execution is known as a task-dependency graph.

Task Dependency Graph (Cont.)

- “It is a **directed acyclic graph** in which node are tasks and the directed edges indicate the dependencies between them”
- The task corresponding to a node can be executed when all tasks connected to this node by **incoming edges** have completed
 - Some tasks **may use data** produced by other tasks and thus may need to wait for these tasks to finish execution

Example of Task Dependence

ID#	Model	Year	Color	Dealer	Price
4523	Civic	2002	Blue	MN	\$18,000
3476	Corolla	1999	White	IL	\$15,000
7623	Camry	2001	Green	NY	\$21,000
9834	Prius	2001	Green	CA	\$18,000
6734	Civic	2001	White	OR	\$17,000
5342	Altima	2001	Green	FL	\$19,000
3845	Maxima	2001	Blue	NY	\$22,000
8354	Accord	2000	Green	VT	\$18,000
4395	Civic	2001	Red	CA	\$17,000
7352	Civic	2002	Red	WA	\$18,000

Execution of the query:

**MODEL = "CIVIC" AND YEAR = 2001 AND
(COLOR = "GREEN" OR COLOR = "WHITE")**

Example of Task Dependence

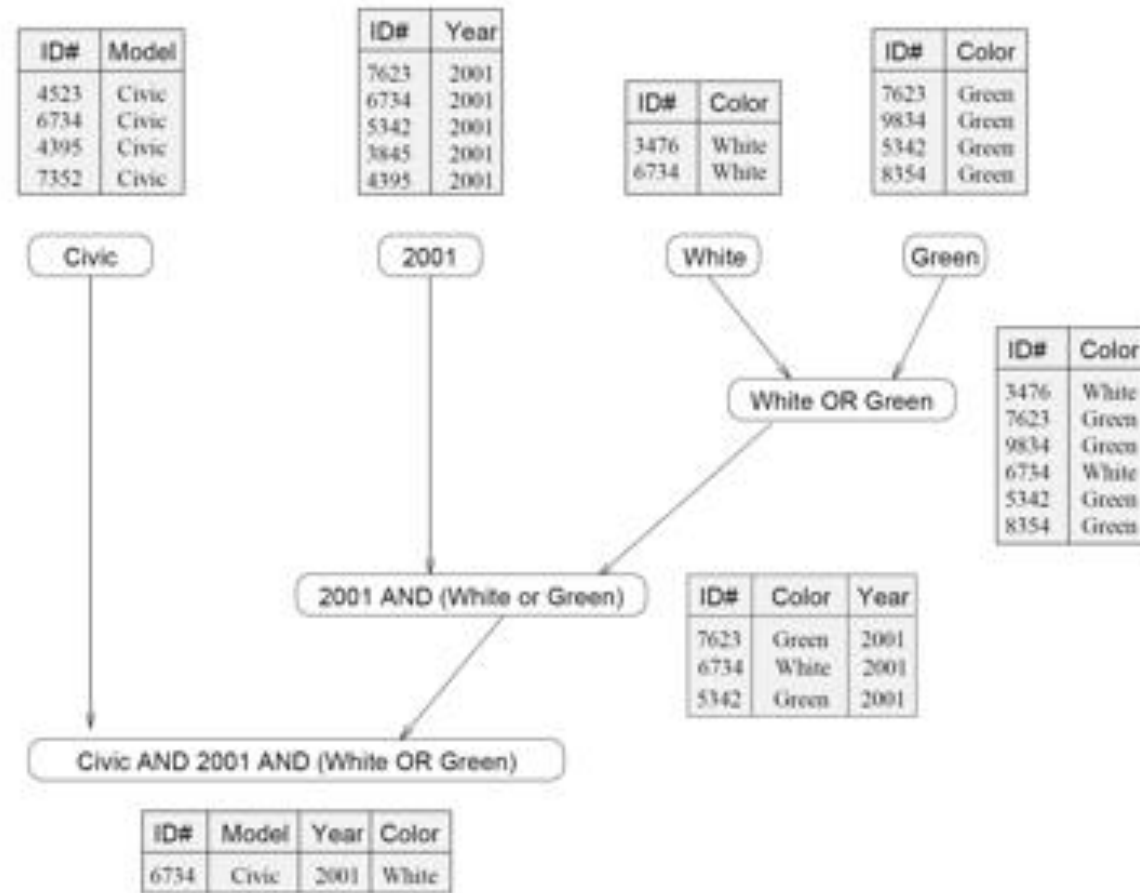


Figure 3.3 An alternate data-dependency graph for the query processing operation.

Granularity

- The **number and sizes of tasks** into which a problem is decomposed determines the granularity of the decomposition
 - Granularity: roughness (means consisting of small grains or particles)
 - A decomposition into a **large number of small tasks** is called fine-grained
 - A decomposition into a **small number of large tasks** is called coarse-grained
- For matrix-vector multiplication, the decomposition would usually be **considered** fine-grained, although coarse-grained could also be an option

Granularity (Cont.)

- Below figure shows a coarse-grained decomposition as each task computes $n/3$ of the entries of the output vector of length n

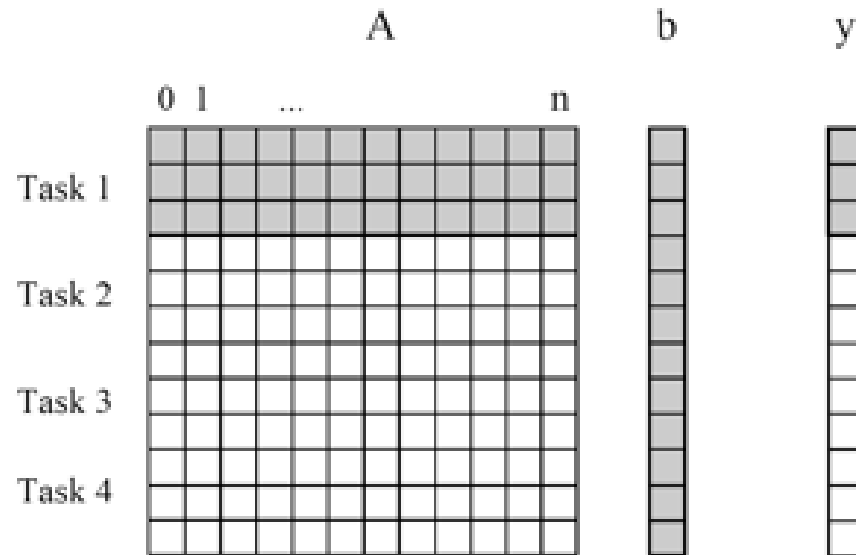


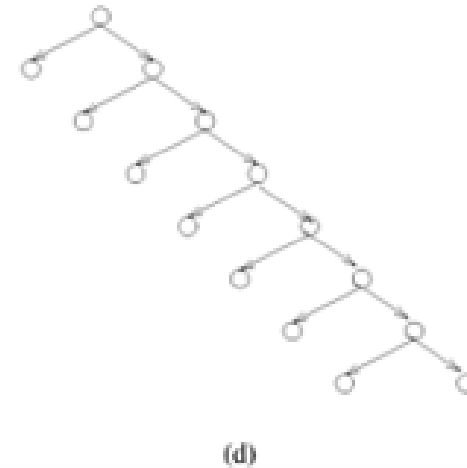
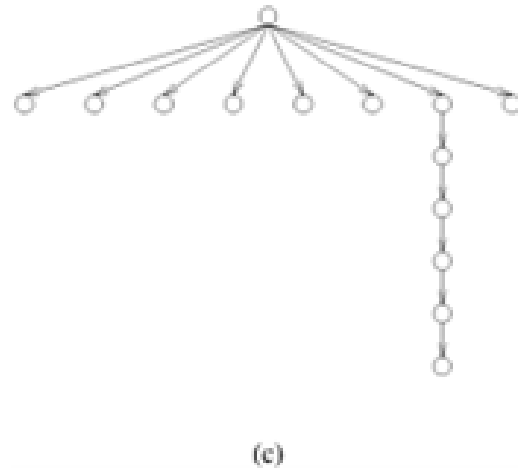
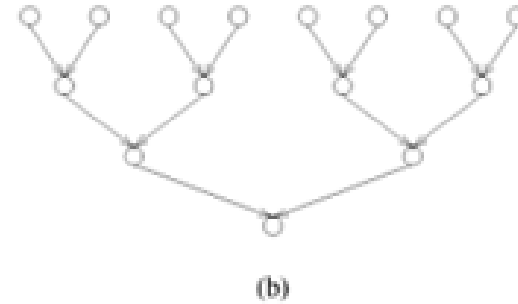
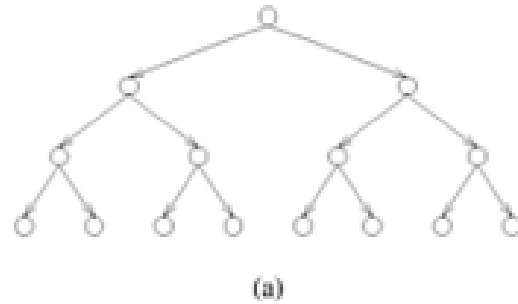
Figure 3.4 Decomposition of dense matrix-vector multiplication into four tasks. The portions of the matrix and the input and output vectors accessed by Task 1 are highlighted.

Maximum Degree of Concurrency

- “The maximum number of tasks that can be executed **simultaneously** in a parallel program at any given time is known as its maximum degree of concurrency.”
- Usually, it is always **less than** total number of tasks due to dependencies.
- E.g., max-degree of **concurrency** in the task-graphs of Figure 3.3 is 4.
- Rule of thumb: For task-dependency graphs that are trees, the maximum degree of concurrency is always **equal** to the number of leaves in the tree

Maximum Degree of Concurrency (Cont.)

- **Determine Maximum Degree of Concurrency?**



Average Degree of Concurrency

- Relatively **better measure** for performance of parallel program
- “The **average number of tasks** that can run concurrently over the entire duration of execution of the program”
- The **ratio** of the total amount of work to the critical-path length
 - Total amount of work: Weight of all the nodes or Tasks
 - Weight of all the nodes is **the size or the amount of work** associated with the corresponding task
 - what is the critical path in a graph?

Critical Path and Critical Path Length

- **Critical Path:** The **longest directed** path between any pair of start and finish nodes is known as the critical path.
- **Critical Path Length:** The **sum of the weights** of nodes along the critical path
- Shorter critical path **favors** a higher average degree of concurrency
- Both, maximum and average degree of concurrency **increases** as tasks become smaller(finier)

Task Interaction Graph

- Depicts **pattern of interaction** between the tasks
- Dependency graphs **show** that how output of first task becomes input to the next level task
- But how the tasks **interact** with each other to access distributed data is depicted by task interaction graphs
- The nodes in a task-interaction graph **represent** tasks
- The edges **connect** tasks that interact with each other
- Example: Dense **matrix-vector** multiplication

Task Interaction Graph (Cont.)

- The edges in a task interaction graph are usually **undirected**
 - But directed edges can be used to indicate the direction of **flow of data**, if it is unidirectional
- The edge-set of a task-interaction graph is a **superset** of the edge-set of the task-dependency graph
 - Tasks 1, 2, 3, 4, 5, and 6 interact each other
 - Task 4 is dependent on the result of task 2
- In database query processing example, the task-interaction graph is the **same** as the task-dependency graph.

Task Interaction Graph (Cont.)

Task Interact Graph (Sparse-matrix multiplication)

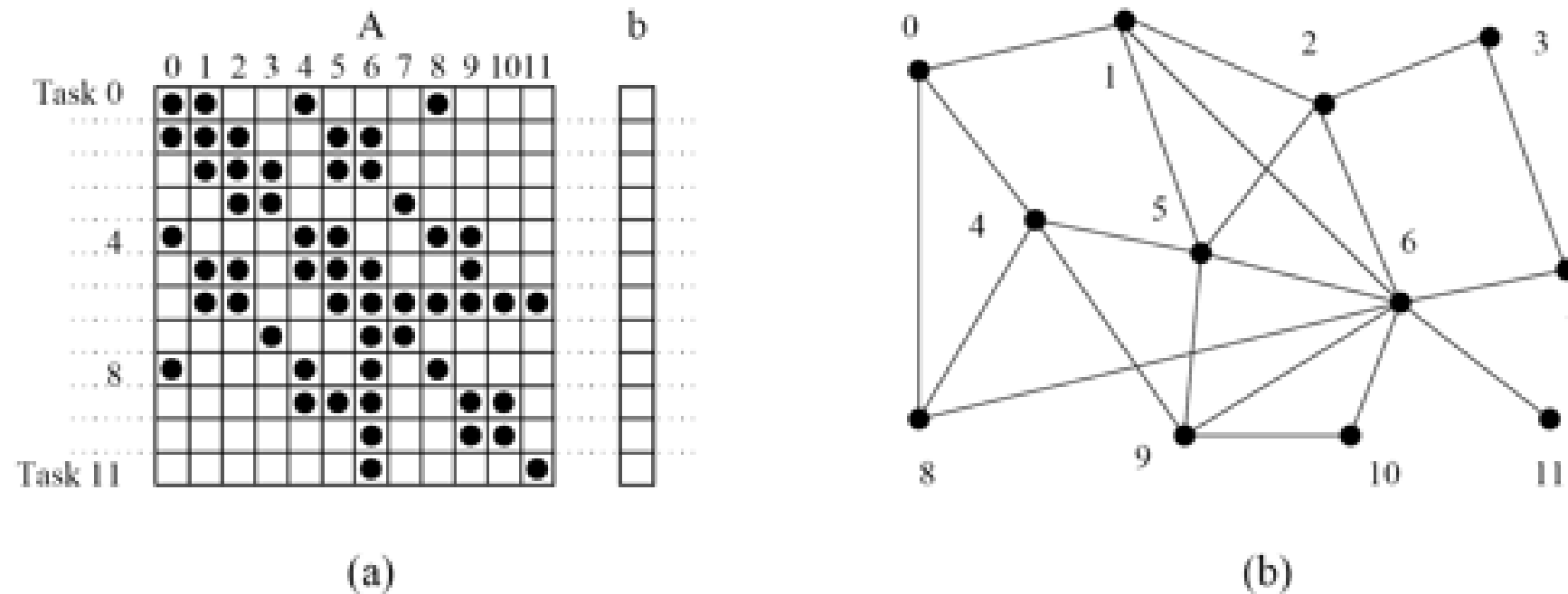
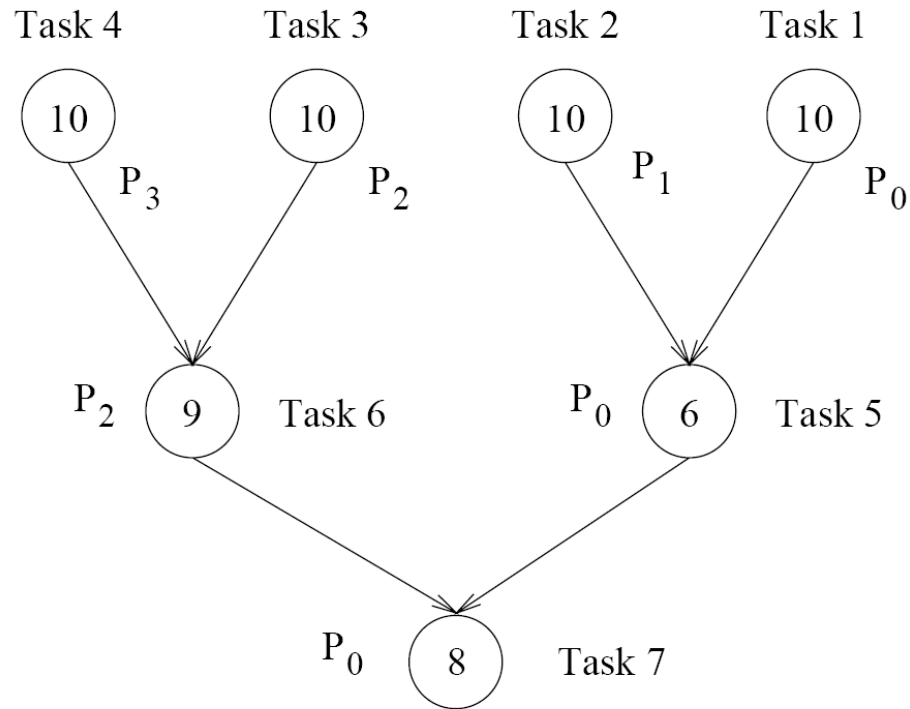


Figure 3.6 A decomposition for sparse matrix-vector multiplication and the corresponding task-interaction graph. In the decomposition Task i computes $\sum_{0 \leq j \leq 11, A[i,j] \neq 0} A[i,j] \cdot b[j]$.

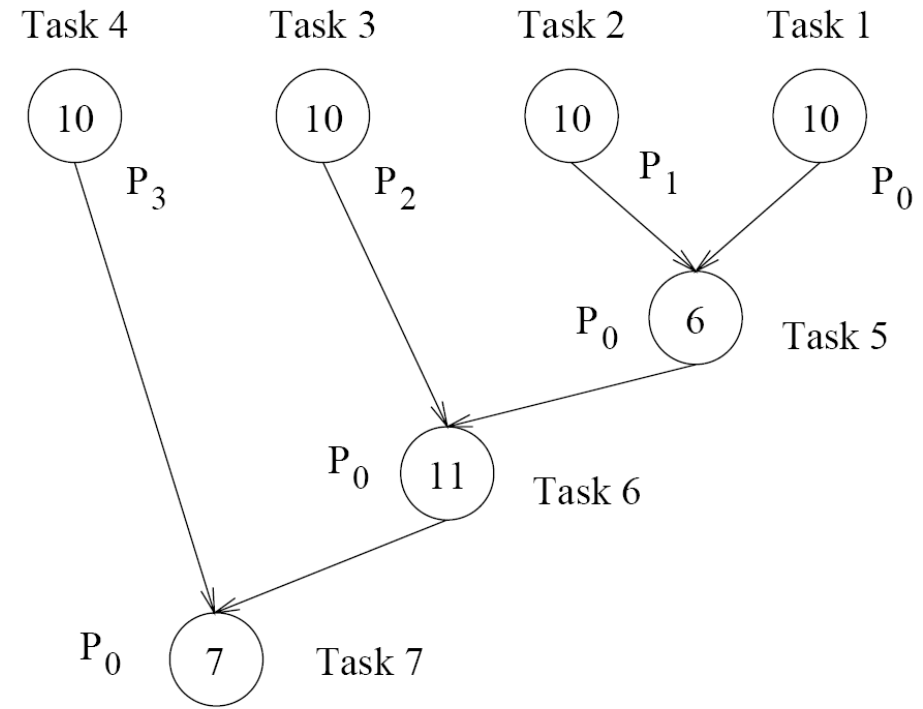
Processes and Mapping

- Logical processing or computing agent that performs tasks is called **process**
- The mechanism by which tasks are assigned to processors for execution is called **mapping**
- Multiple tasks can be mapped on a **single** processor
- Independent task should be mapped onto **different** processors
- Map tasks with **high mutual-interactions** onto a single processor
- A parallel program must have several processors active and simultaneously working on different tasks to gain a significant **speedup** over the sequential program

Processes and Mapping (Cont.)



(a)



(b)

Figure 3.7 Mappings of the task graphs of Figure 3.5 onto four processes.

Processes vs Processors

- Processes are **logical computing agents** that perform tasks
- Processors are the **hardware units** that physically perform computations
- Depending on the problem, multiple processes can be **mapped** on a single processor
- But, in most of the cases, there is **one-to-one correspondence** between processors and processes

Summary

- **Steps in Parallel Algorithm Design**

- Identification - Identification of parallel portion in the program
- Mapping - Mapping concurrent tasks onto multiple processors
- Data Partitioning - Distribution of input, output, and intermediate data associated with the program
- Defining Access Protocol - Managing accesses to data shared by multiple processors

- **Parallel Computing Example**

- IBM Deep Blue machine beats Chess World Champion Kasparov using 30 nodes of 120 MHZ microprocessors

- **Task Decomposition**

- Dividing of a process into multiple tasks that may be arbitrary sizes

Summary (Cont.)

- **Multiplication of a Dense Matrix with a Vector**

- Vector Multiplication $n \times 1$

- Matrix Multiplication $n \times n$

- ✓ Independent computation, no task dependency graph

- **Task Dependency Graph**

- An abstraction used to express dependencies among tasks and their relative order of execution

- A directed acyclic graph in which nodes are tasks and the directed edges indicate the dependencies between them

- **Example of Task Dependence**

- Execution of a database query

Summary (Cont.)

- **Granularity**

- **Number and sizes of tasks into which a problem is decomposed determines the granularity of the decomposition**
 - ✓ A decomposition into a large number of small tasks is called fine-grained
 - ✓ A decomposition into a small number of large tasks is called coarse-grained

- **Maximum and Average Degree of Concurrency**

- **Maximum number of tasks that can be executed simultaneously in a parallel program at any given time is known as its maximum degree of concurrency**
- **Average number of tasks that can run concurrently over the entire duration of execution of the program**

Summary (Cont.)

- **Critical Path and Critical Path Length**

- **Critical Path:** The longest directed path between any pair of start and finish nodes.
- **Critical Path Length:** The sum of the weights of nodes along the critical path

- **Task Interaction Graph**

- **Dependency graphs** show that how output of first task becomes input to the next level task
- **But how the tasks interact with each other to access distributed data is depicted by task interaction graphs**

Summary (Cont.)

- **Processes and Mapping**

- Logical processing or computing agent that performs tasks is called process
- The mechanism by which tasks are assigned to processors for execution is called mapping

- **Processes vs Processors**

- Processes are logical computing agents that perform tasks
- Processors are the hardware units that physically perform computations

Additional Resources

- **Introduction to Parallel Computing** by Ananth Grama and Anshul Gupta

➤ **Chapter 3. Principles of Parallel Algorithm Design**

- ✓ **Section 3.1: Preliminaries**

Questions?

Class Activity